

# WEAVING CI/CD INTO FABRIC

Implementing CI/CD Best Practices with  
fabric-cicd

Vienna, 2026-05-07

Stefan Mikic

# Our agenda for today

**01** A bit about Cloudflight | paiqo

**02** And also, a bit about me :)

**03** Our Problem

**04** What is CI/CD

**05** Available Options

**06** Our Decision

**07** Current Design

**08** What changed up until v1?

**09** Demo

# 01

A bit about Cloudflight | paiqo

# Cloudflight and paiqo

**+ 1,000 Clients**

in the areas of manufacturing, logistics, retail, public sector, mobility, healthcare, and aerospace

**> 2,000 Projects**

with solid expertise in the fields of software development, data platforms, and AI

**+ 700 Engineers**

software developers, data scientists, data engineers, cloud infrastructure engineers, and more

## Solution areas

Tech Advisory  
& AI Strategy

AI-augmented  
Software Development

Agentic & Classic AI

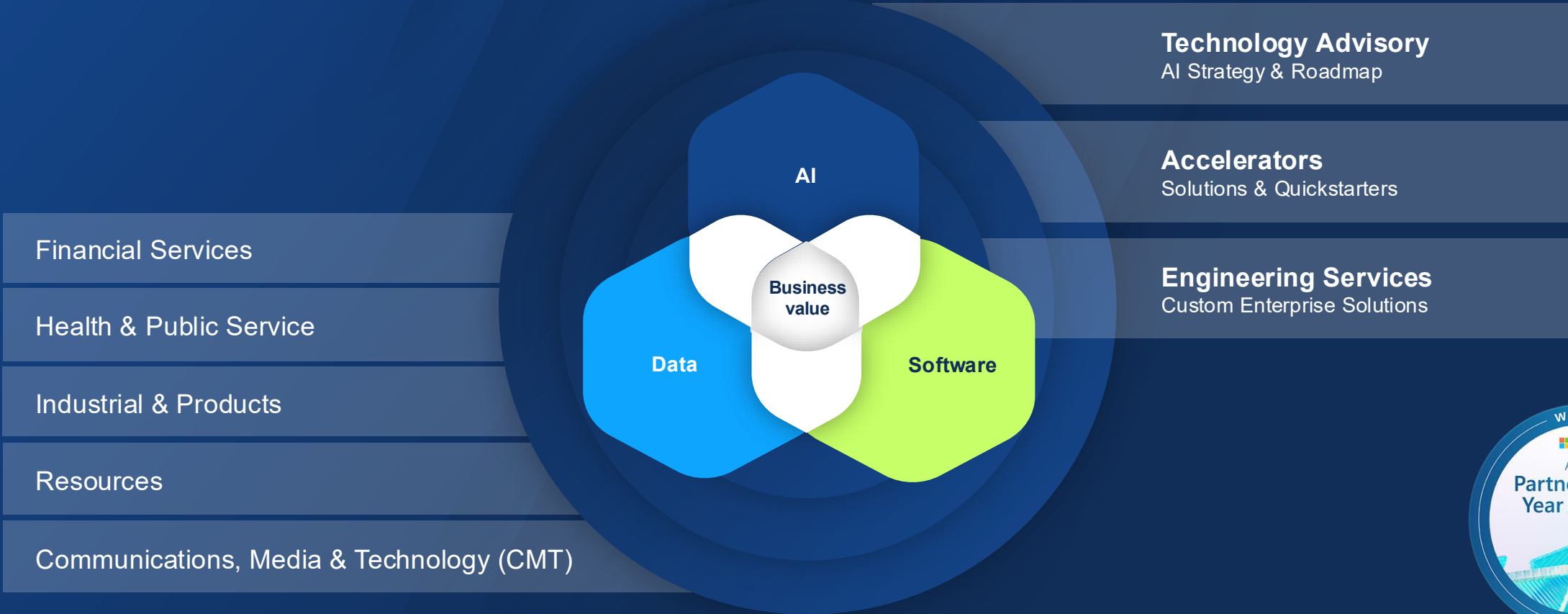
Agentic Data &  
Context Platforms



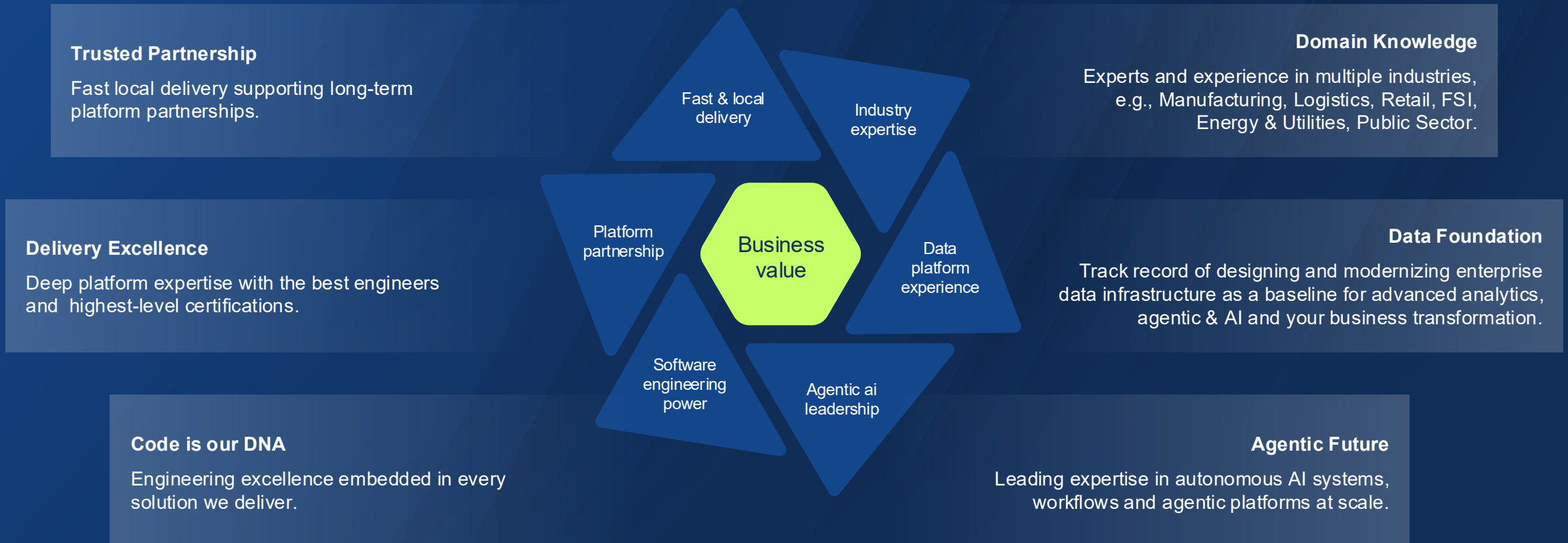
Selected clients



# Our solution portfolio



# Fast Results. Real Business Value.



# Our technological expertise lets you build the **digital solutions** you always wanted



# 02

And also, a bit about me :)

# About me :)



**Stefan Mikic**

Senior Data Engineer 🥕🥦🍅 @ Cloudflight

<https://smikic.com>

# 03

## Our Problem

# Our Problem

**1**

## No automated Pipelines

Semantic models and reports across domain workspaces have no automated deployment every update requires manual effort.

**2**

## Manual Git actions slow delivery

Deployments depend on manual branch syncs and Git actions.

Updates accumulate and ship in batches rather than continuously.

**3**

## Human error & inconsistencies

Without automation, environments can drift.

What lives in TEST may silently differ from PRD or domain workspace content.

**4**

## Friction for new report additions

Onboarding a new report to a domain workspace required touching deployment config manually

no single source of truth for what deploys where.

# 04

What is CI/CD?

# What is CI/CD

## Continuous Integration

Merge early, test often

- Developers push changes to a shared repository frequently
- Every change is automatically validated → linting, tests, schema checks
- Catch problems early, before they compound
- One source of truth for what the system looks like

## Continuous Delivery

Ship reliably, on demand

- Validated changes are automatically deployed to various environments
- Rollback fast → every deployment is traceable
- Environments stay consistent and reproducible

# Why it matters in Data Engineering

1

## Faster iteration

Pipeline changes, model updates, and report fixes reach users in minutes, not days of manual coordination.

2

## Fewer production incidents

Automated validation catches broken schemas, missing dependencies, and config mismatches before deployment.

3

## Full auditability

Every change is tracked in Git with who, what, when, and why

4

## Environment consistency

DEV, TEST, and PROD stay in sync. No more "it works on my machine" → every environment is built the same way.

5

## Scale without chaos

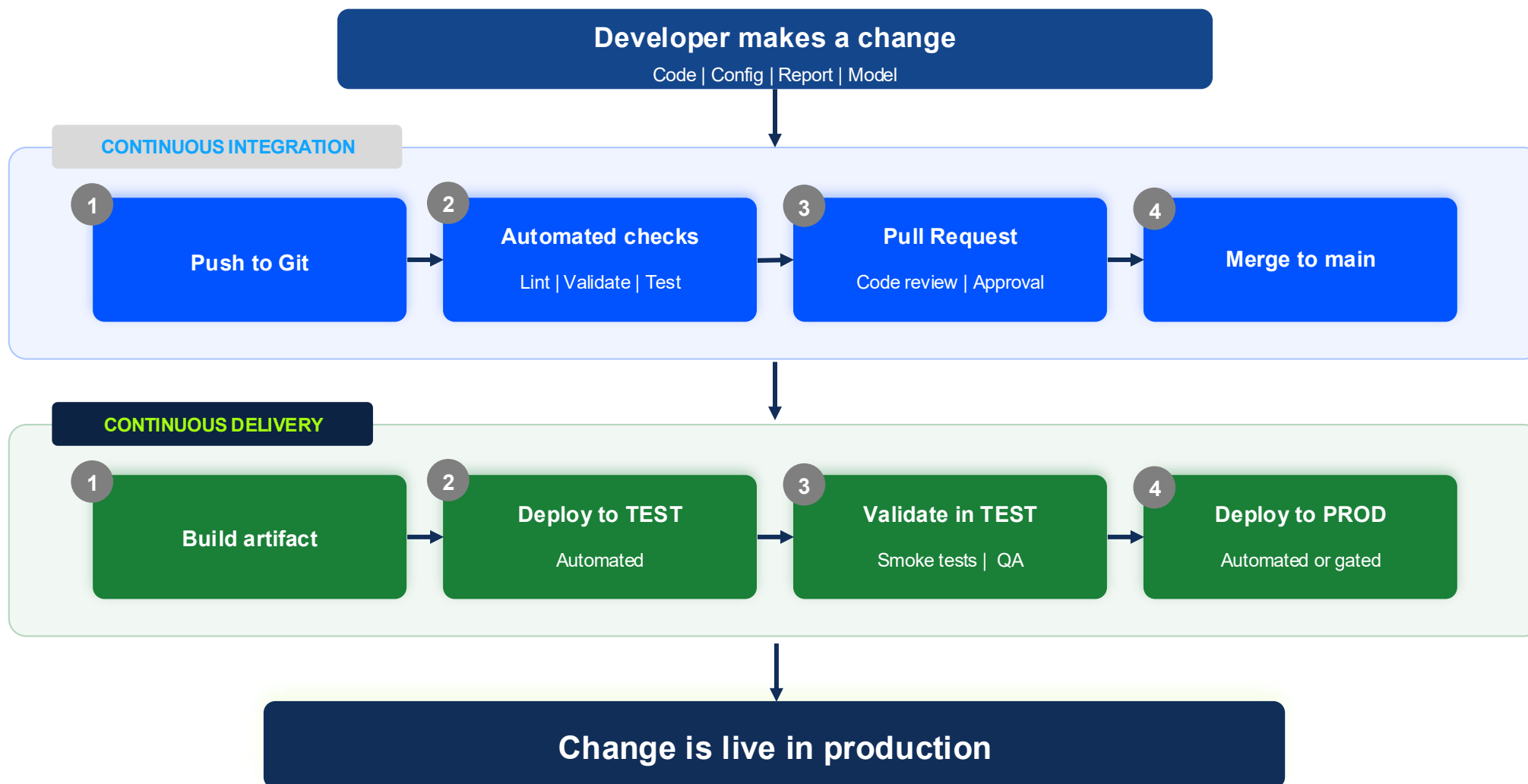
As teams grow and artifacts multiply, manual deployment becomes a bottleneck. CI/CD scales with the platform.

6

## Reliable rollbacks

If something goes wrong, revert to the last known-good state in → minimizes the need to manual search for root causes

# A typical CI/CD Workflow



# 05

## Available Options

# Available Options

## Fabric Deployment Pipelines

### ✓ Pros

- Native Fabric UI
- Up to 10 configurable stages

### ✗ Cons

- Not source-controlled
- Linear promotion only (DEV → TEST → PRD)

## fabric-cicd (Python library)

### ✓ Pros

- Code-first, fully Git-managed
- Works with any CI/CD system
- 24+ item types incl. Preview
- Flexible routing to any workspace

### ✗ Cons

- Full redeployment
- Requires Software Engineering Fundamentals

## Direct REST API

### ✓ Pros

- Maximum control → any item type
- Incremental updates possible
- Custom retry / rollback logic

### ✗ Cons

- Not source-controlled
- Linear promotion only (DEV → TEST → PRD)

# 06

## Our Decision

# Our Decision

## Everything in Git

Config, item lists and parameter mappings, version-controlled, auditable, reproducible.

## Works with our CI/CD setup

Plugs directly into Azure DevOps with no special Fabric-native tooling required.

## Handles our routing complexity

Same item, different semantic model per domain workspace (ESG, Finance...).

## Broad item type coverage

Notebooks, Lakehouses, DataPipelines, Semantic Models and Reports  
→ all in one tool.

## Active development / community

Microsoft-maintained open-source library - v1 released as stable GA.



**Applies SWE best practices in Fabric:**

**Staging areas, automated CI/CD,  
version control & complete isolation  
of PROD.**

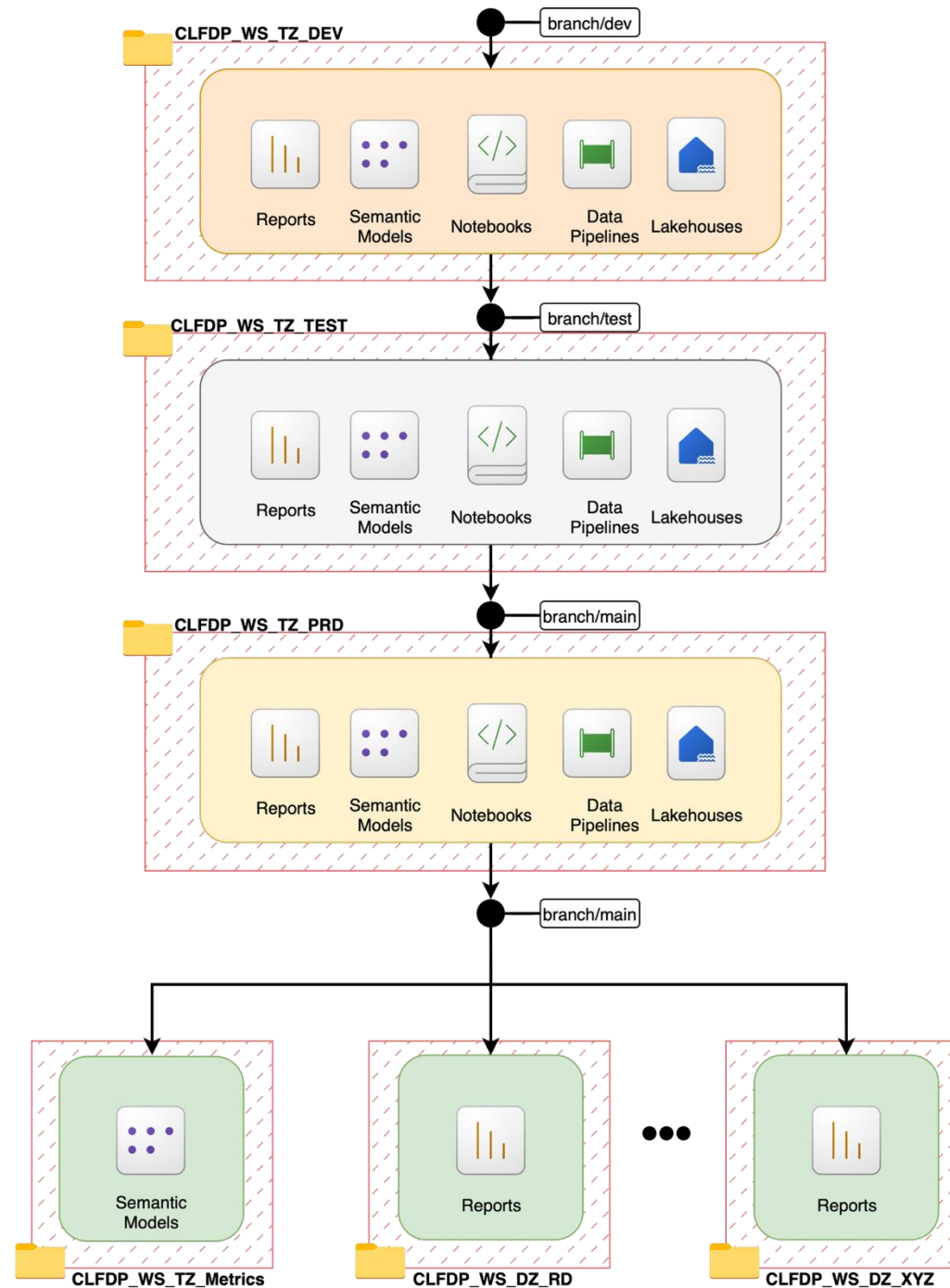
---

- Our Learnings, presentation v0.3

# 07

## Current Design

# Current Design



# Current Design

## PBIR rewrite script

semantic\_model\_binding didn't exist yet, manually patching .pbir was the only way to bind reports.

## Jinja2 parameter template

Dynamic variables weren't available, Jinja2 was the pragmatic solution for environment-specific IDs.

## Feature flags for standard features

items\_to\_include required explicit opt-in via append\_feature\_flag().  
Now default in v1.

```
1 devops/  
2 |— Dockerfile  
3 |— fabric_deployment.py  
4 |— helpers  
5 |   |— pbir_binding.py  
6 |— Makefile  
7 |— parameter.yml  
8 |— pipelines  
9 |   |— deployment.env  
10 |   |— deployment.yml  
11 |— poetry.lock  
12 |— pyproject.toml  
13 |— workspace_parameters.json  
14
```

# 08

What changed up until V1

# What changed up until v1?

1

## semantic\_model\_binding

Report-to-model wiring in parameter.yml.  
Replaces custom binding entirely.

2

## Dynamic variables

`$workspace.$id` and `$items.<Type>.<Name>.$id`  
resolve at runtime → no Jinja2.

3

## No feature flags needed

`items_to_include`, env var replacement and others are now default behaviour.

4

## key\_value\_replace (JSON Path)

Target exact JSON keys in pipeline/notebook files, more precise than `find_replace`.

5

## Config file (config.yml)

Deployment behaviour (item types, filters) can move to YAML

6

## workspace\_name support

FabricWorkspace accepts `workspace_name` or `workspace_id` → cleaner scripts

# 09

DEMO TIME